Machine Learning Complete Notes

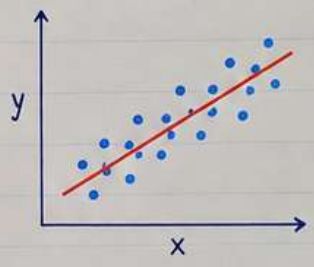
Machine Learning

COMPLETE NOTES

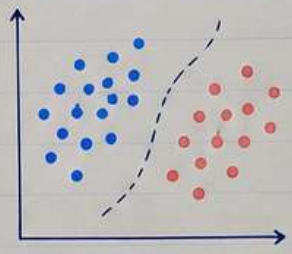
From Basics to Advanced

Data → Preprocessing → Feature Engineering → Model Training → Evaluation → Prediction

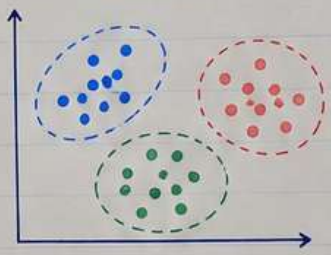
Regression



Classification



Clustering



Topics Covered

1. Machine Learning Fundamentals
2. Data & Problem Understanding
3. Data Preprocessing
4. Exploratory Data Analysis
5. Feature Engineering
6. Linear Regression
7. Regression Algorithms
8. Regression Evaluation
9. Logistic Regression
10. Classification Algorithms
11. Classification Evaluation
12. Decision Trees
13. Ensemble Learning
14. Support Vector Machine & KNN
15. Unsupervised Learning
16. Dimensionality Reduction
17. Model Selection & Validation
18. Hyperparameter Tuning
19. Advanced ML Concepts
20. ML Interview Master + Cheat Sheet

Next Chapters in the Series

Python → **Machine Learning** → Deep Learning → Neural Networks (CNN, RNN) → Transformers → GenAI & LLMs

Machine Learning Complete Notes

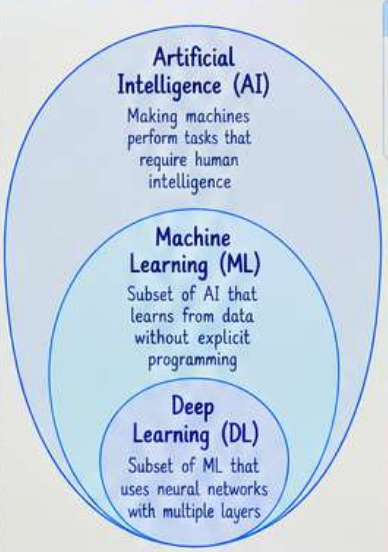
1 Machine Learning Fundamentals

1. What is Machine Learning?

- Machine Learning (ML) is a branch of Artificial Intelligence that enables systems to learn patterns from data and make decisions or predictions without being explicitly programmed.
- It uses algorithms to find patterns, improve from experience and generalize to new, unseen data.
- The performance of an ML model improves as it is trained on more data.

Goal: To enable computers to learn from data and make accurate predictions or decisions without explicitly programmed rules.

2. AI vs ML vs DL



Artificial Intelligence (AI)

- Broad field of making machines perform intelligent tasks.
- Includes rule-based systems, ML, DL, planning, reasoning, etc.

Machine Learning (ML)

- Subset of AI.
- Learns patterns from data without explicit programming.
- Uses statistical and mathematical models.

Deep Learning (DL)

- Subset of ML.
- Uses artificial neural networks with multiple layers (deep networks).
- Works very well for large and complex datasets (images, text, etc.).

3. Why Machine Learning?

- Can learn from data and improve automatically.
- Finds patterns in large and complex datasets.
- Useful for making predictions and data-driven decisions.
- Handles complex and non-linear relationships.
- Widely used in real-world applications (recommendations, fraud detection, image recognition, etc.).

4. Types of Machine Learning

- Machine Learning can be broadly classified into three types:

Supervised Learning

- Learns from labeled data (input and output pair).
- Used for prediction tasks.
- Examples: classification, regression.

Unsupervised Learning

- Learns from unlabeled data (only input).
- Finds hidden patterns or structures in data.
- Examples: clustering, dimensionality reduction.

Reinforcement Learning

- Learns through interaction with an environment.
- Improves by receiving rewards or penalties.
- Examples: game playing, robotics, navigation.

5. Supervised Learning

- Uses labeled data (input features and known output).
- Learns a mapping from input to output.
- Main tasks: Classification and Regression.
- Examples: email spam detection, house price prediction.

Labeled Data	
Input (X)	Output (y)
Feature 1	Class A
Feature 2	Class B
Feature 3	Class A
...	...

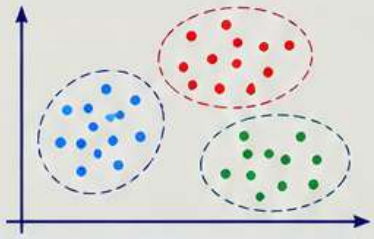
→ **Learning Algorithm** →

Prediction

Class A / Class B
(or a continuous value)

6. Unsupervised Learning

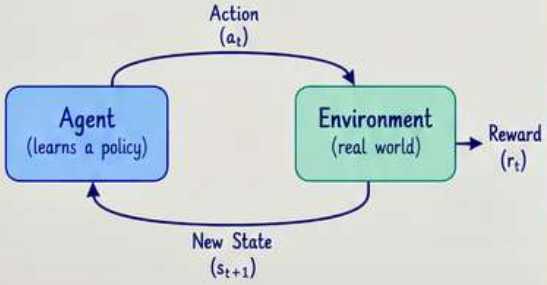
- Uses unlabeled data (only input features).
- Finds hidden patterns or groups in the data.
- Main tasks: Clustering, Dimensionality Reduction, Association.
- Examples: customer segmentation, anomaly detection.



The algorithm finds natural groupings or structures in the data without any labels.

7. Reinforcement Learning

- Learns by interacting with an environment.
- Makes decisions (actions) and receives rewards or penalties.
- Goal is to maximize the cumulative reward over time.
- Examples: game playing (e.g., chess, Atari), robotics, self-driving cars.



8. Machine Learning Workflow

- A general pipeline followed in most ML projects.
- Each step is important for building a good model.

Data → **Preprocessing** → **Features** → **Model** → **Training** → **Evaluation** → **Prediction**

- Collect data
- Understand the problem
- Explore data (EDA)

- Handle missing values
- Remove outliers
- Clean data
- Encode categorical variables

- Feature engineering
- Feature selection
- Feature scaling

- Choose algorithm
- Set hyper-parameters

- Train on training data
- Use learning algorithm

- Evaluate using metrics (Accuracy, MAE, MSE, etc.)
- Cross-validation

- Make predictions on new unseen data

Machine Learning Complete Notes

2 Data & Problem Understanding

1. Dataset

- A dataset is a collection of data points used to train and evaluate a machine learning model.
- Each row (record) represents an observation and each column represents a feature (variable).
- Datasets can be numerical, categorical, text, images, etc.

Example Dataset (House Price)

Size (sqft)	Bedrooms	Location	Price (₹)
1200	2	Urban	5500000
1500	3	Urban	7000000
800	2	Suburban	3000000
2000	4	Urban	9500000

2. Features

- Features (or input variables) are the independent variables used to make predictions.
- They contain information about the data.
- Features can be numerical, categorical or text-based.

Examples:
- Size, number of bedrooms, location, age of house, etc.
- In the above dataset, Size, Bedrooms and Location are features.

3. Target

- The target (or output variable) is the dependent variable that we want to predict.
- It is the value the model learns to estimate.

Example:
- In the house price dataset, Price is the target.
- In a classification problem, the target can be a class (e.g., spam or not spam).

4. Observations

- Each row in the dataset is called an observation (or sample).
- It represents a single data point with values for all features and the target.

Example:
- In the house price dataset, each row (e.g., 1200, 2, Urban, 5500000) is one observation.
- If the dataset has 1000 rows, it has 1000 observations.

5. Independent vs Dependent Variables

- Independent variables (features): Input variables used for prediction. (X)
- Dependent variable (target): Output variable to be predicted. (y)

Independent Variables (Features) X → **Dependent Variable (Target) y**

Example: Size, Bedrooms, Location → Price

6. Numerical vs Categorical Data

- Numerical data: Represents quantities, can perform mathematical operations.
- Categorical data: Represents categories or groups.

Numerical Data

- Continuous or discrete values.
- Examples: age, height, price, temperature, number of rooms.
- Can be further divided into:
  - Continuous (any value) e.g., 12.5, 18.7
  - Discrete (countable values) e.g., 1, 2, 3

Categorical Data

- Represents categories or labels.
- Examples: gender, location, color, product type.
- Types:
  - Nominal (no order) e.g., Male, Female, Urban, Rural
  - Ordinal (ordered categories) e.g., Low, Medium, High

7. Structured vs Unstructured Data

- Structured data: Organized in a fixed format (rows and columns). E.g., Excel files, relational databases.
- Unstructured data: Does not have a fixed format. E.g., text, images, audio, videos.

Structured Data

ID	Name	Age	City
1	A	25	Trivandrum
2	B	30	Kochi
3	C	22	Delhi

- Easy to store and analyze.
- Commonly used in traditional ML models.

Unstructured Data

Text · Images · Audio · Videos

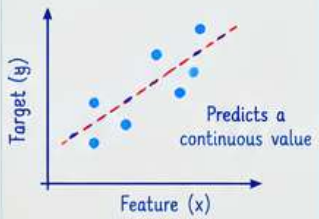
- No fixed format.
- Requires special processing (e.g., NLP, Computer Vision).
- Used in advanced ML models.

8. Regression vs Classification Problem

- **Regression:** Predicts a continuous numerical value.
- **Classification:** Predicts a discrete category or class.

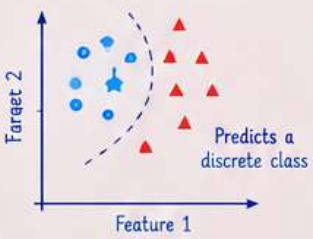
Regression

- Target variable is continuous (numerical).
- Examples: house price prediction, stock price prediction, temperature forecasting.



Classification

- Target variable is categorical (discrete).
- Examples: email spam detection, disease prediction, image classification.



9. Business Problem → ML Problem

- In real-world applications, the problem usually comes from a business or domain need.
- We need to convert the business problem into a machine learning problem by identifying the target, features and type of model.

Example 1: House Price Prediction

- **Business Problem:** Estimate the price of a house based on its features.
- **ML Problem:** Regression problem to predict a continuous value (price).
- **Features:** Size, bedrooms, location, age, etc.
- **Target:** House price (continuous).

Example 2: Email Spam Detection

- **Business Problem:** Identify whether an email is spam or not.
- **ML Problem:** Classification problem to predict a class (spam or not spam).
- **Features:** Text content, word frequency, sender information, etc.
- **Target:** Spam / Not Spam (categorical).

Machine Learning Complete Notes

3 Data Preprocessing

- Data preprocessing is the process of cleaning and transforming raw data into a suitable format for building machine learning models.
- It helps improve data quality, remove noise and ensures better model performance.
- The steps include handling missing values, duplicates, outliers, data types, encoding categorical variables, feature scaling, train-test split and avoiding data leakage.

1. Missing Values

- Missing values occur when some data points are not available.
- They can affect the model performance.
- Common ways to handle missing values:

Method	When to Use
Remove rows	When missing values are few.
Fill with mean/median	For numerical data.
Fill with mode	For categorical data.
Predict missing values	Advanced method (e.g., using models).

```
# Fill missing values
df['age'] = df['age'].fillna(df['age'].median())
df['city'] = df['city'].fillna(df['city'].mode()[0])
```

2. Duplicates

- Duplicate rows are repeated records in the dataset.
- They can lead to biased model training.
- We can remove duplicate rows using:

```
# Remove duplicate rows
df = df.drop_duplicates()
```

Before Removing Duplicates

ID	Name	Age
1	A	25
2	B	30
2	B	30
3	C	28

→

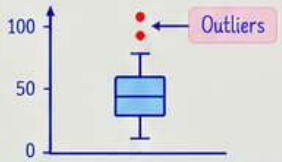
After Removing Duplicates

ID	Name	Age
1	A	25
2	B	30
3	C	28

3. Outliers

- Outliers are data points that are significantly different from the rest of the data.
- They can affect the model performance, especially in algorithms like linear regression.
- Methods to detect and handle outliers:

Method	Description
Z-score	Identify points with $\|z\| > 3$.
IQR (Interquartile Range)	Use 1.5 × IQR rule.
Visualization	Use boxplot or scatter plot.
Handling	Remove, transform (log), or cap (winsorization).



IQR method:

Lower bound = $Q_1 - 1.5 \times IQR$

Upper bound = $Q_3 + 1.5 \times IQR$

4. Data Types

- Data can be numerical or categorical.
- Checking and converting data types helps in proper preprocessing.

Data Type	Description
int	Integer values (e.g., 1, 2, 3)
float	Decimal values (e.g., 1.5, 2.3)
object	String or categorical values
bool	Boolean values (True/False)
datetime	Date and time values

```
# Check data types
df.dtypes

# Convert data type
df['age'] = df['age'].astype(int)
```

5. Encoding Categorical Variables

- Machine learning models require numerical input, so categorical variables need to be converted into numbers.
- Common encoding techniques:

Method	Description	Example
Label Encoding	Assign a unique number to each category.	Red → 0, Blue → 1
One-Hot Encoding	Create binary columns for each category.	Color → [1, 0], [0, 1]
Ordinal Encoding	For ordered categories.	Low → 0, Medium → 1, High → 2
Frequency Encoding	Replace with frequency of each category.	City → number of occurrences
Target Encoding	Replace with target mean (used carefully).	Advanced technique.

6. Feature Scaling

- Feature scaling helps bring all features to the same scale.
- It is important for algorithms like KNN, SVM, and Neural Networks, which are sensitive to the scale of data.
- Common methods:

Method	Formula	Description
Standardization (Z-score)	$z = \frac{x - \mu}{\sigma}$	Mean = 0, Std = 1. Used when data is normally distributed.
Min-Max Scaling	$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$	Scales data to [0, 1]. Used when the range is known.

7. Train-Test Split

- The dataset is divided into training and testing sets.
- Training set is used to train the model and test set is used to evaluate its performance on unseen data.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
  X, y, test_size=0.2, random_state=42)
```

8. Data Leakage

- Data leakage happens when information from the test set is used during training, either directly or indirectly.
- It leads to unrealistically high performance and poor generalization.
- To avoid data leakage:
- Perform preprocessing (scaling, encoding, etc.) after splitting the data.
- Do not use target-related information in features.
- Use pipelines to ensure correct order of preprocessing and modeling.
- Be careful with time-based data (use past data only).

Machine Learning Complete Notes

4 Exploratory Data Analysis

- Exploratory Data Analysis (EDA) is the process of analyzing and visualizing data to understand its main characteristics, find patterns, detect anomalies and get insights before building a machine learning model.
- EDA helps in making better decisions for data preprocessing, feature engineering and model selection.

1. Univariate Analysis

- Analyzes one variable at a time.
- Helps to understand the distribution, central tendency, spread and presence of outliers.
- Common for both numerical and categorical data.

Examples:
- Mean, median, mode, std deviation (numerical)
- Value counts, bar plots (categorical)
- Histograms, box plots to understand distribution

2. Bivariate Analysis

- Analyzes the relationship between two variables.
- Helps to understand how one variable changes with another variable.
- Commonly used to identify correlations and patterns.

Examples:
- Scatter plot (numerical vs numerical)
- Box plot (numerical vs categorical)
- Bar plot / Count plot (categorical vs numerical)
- Correlation analysis

3. Multivariate Analysis

- Analyzes the relationship between three or more variables.
- Helps to find complex patterns and interactions.
- Useful for feature selection and understanding high-dimensional data.

Examples:
- Pair plot (multiple numerical variables)
- Heatmap (correlation between multiple variables)
- 3D scatter plot
- Dimensionality reduction techniques (PCA)

4. Distribution

- Shows how the values of a variable are spread.
- Helps to understand the shape, center and spread of data.
- Common distribution shapes: normal, skewed (left or right), uniform, etc.



5. Correlation

- Measures the strength and direction of the linear relationship between two numerical variables.
- Correlation value ranges from -1 to 1.

Correlation Value	Interpretation
1	Perfect positive correlation
0.5 to 0.99	Strong positive correlation
0.1 to 0.49	Weak positive correlation
0	No correlation
-0.1 to -0.49	Weak negative correlation
-0.5 to -0.99	Strong negative correlation
-1	Perfect negative correlation

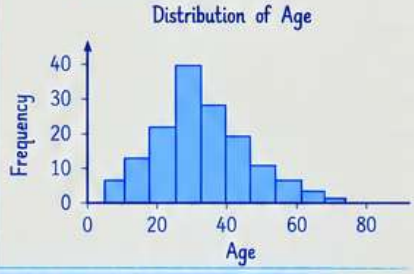
6. Outlier Detection

- Outliers are data points that are significantly different from the rest of the data.
- They can affect model performance and need to be identified and handled.

Common methods:
- Box plot (IQR method)
- Z-score
- Scatter plot
- Visual inspection

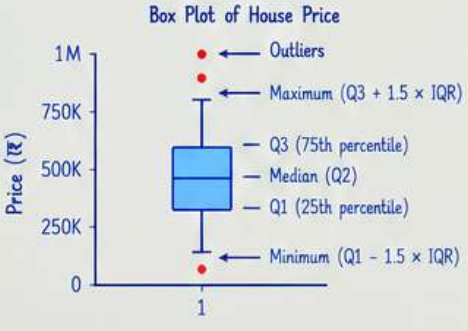
7. Histograms

- A histogram shows the frequency distribution of a numerical variable using bins.
- Helps to understand the shape, spread and outliers.



8. Box Plots

- Box plots show the distribution of a numerical variable using five summary statistics: minimum, Q1, median, Q3 and maximum.
- Helps to identify outliers.



9. Scatter Plots

- Scatter plots show the relationship between two numerical variables.
- Helps to identify patterns, trends, clusters and outliers.



10. Correlation Heatmap

- A heatmap shows the correlation between multiple numerical variables using color coding.
- Helps to quickly identify strong positive or negative correlations.



Machine Learning Complete Notes

5 Feature Engineering

- Feature Engineering is the process of creating, transforming and selecting the most relevant features from raw data to improve model performance.
- It helps the model understand the underlying patterns, reduce noise and capture important information from the data.

1. Feature Creation

- Creating new features from existing ones using domain knowledge or mathematical operations.
- Helps to capture additional information that may be useful for the model.

Examples:
- Creating ratios, differences or interactions between features.
- Domain-based features (e.g., price per unit, age groups, distance, etc.).

Example: Creating New Features

Original Features	New Feature
total_price, quantity	price_per_unit = total_price / quantity
date	day_of_week, month, is_weekend
height, weight	BMI = weight / (height^2)

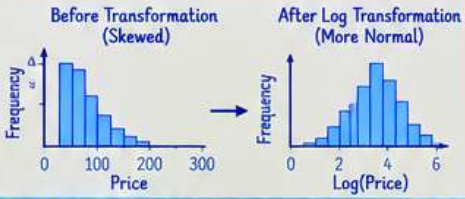
2. Feature Transformation

- Transforming existing features to improve their distribution, reduce skewness or make them more suitable for the model.
- Helps in handling non-linear relationships and improving model performance.

Common transformations:
- Log transformation
- Square root transformation
- Box-Cox / Yeo-Johnson transformation
- Polynomial features
- Binning (discretization)

Example: Log Transformation

```
import numpy as np
df['log_price'] = np.log1p(df['price'])
```



3. Feature Selection

- Selecting the most relevant features and removing irrelevant or redundant features.
- Helps to reduce overfitting, improve model performance and reduce computational cost.

Methods:
- Filter methods (correlation, chi-square, ANOVA, etc.)
- Wrapper methods (RFE, stepwise selection, etc.)
- Embedded methods (Lasso, Tree-based feature importance, etc.)

Example: Select Top Features (using correlation)

```
import pandas as pd
corr = df.corr()['target'].abs()
top_features = corr.sort_values(
    ascending=False).head(5)
print(top_features)
```

4. Binning (Discretization)

- Converting a continuous numerical feature into discrete bins or categories.
- Helps to reduce the effect of outliers. and capture non-linear relationships.

Methods:
- Equal width binning
- Equal frequency binning (quantile)
- Domain-based binning (custom ranges)

Example: Binning using pd.cut()

```
df['age_group'] = pd.cut(
   df['age'],
   bins=[0, 18, 30, 50, 100],
   labels=['Child', 'Young', 'Adult', 'Senior']
)
```

Example Output

Age	Age Group
12	Child
25	Young
42	Adult
67	Senior

5. Encoding

- Converting categorical variables into numerical format so that the model can understand them.
- Different encoding techniques are used depending on the type and number of categories.

Method	Description	Example
Label Encoding	Assign a unique number to each category.	Red → 0 Blue → 1 Green → 2
One-Hot Encoding	Create binary columns for each category.	Red → [1, 0, 0] Blue → [0, 1, 0]
Ordinal Encoding	Used for ordered categories.	Low → 0 Medium → 1 High → 2
Target Encoding	Replace with target mean (use carefully).	Category → mean target value
Frequency Encoding	Replace with frequency of each category.	City A → 50 City B → 30

6. Scaling

- Bringing all numerical features to the same scale.
- Helps models like KNN, SVM and Neural Networks which are sensitive to feature scale.
- Common scaling methods:

Method	Formula	Use Case
Standardization (Z-score)	$z = \frac{x - \mu}{\sigma}$	When data is normally distributed (mean = 0, std = 1).
Min-Max Scaling	$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$	Scales data to [0, 1]. When range is known.
Robust Scaling	$x' = \frac{x - \text{median}}{\text{IQR}}$	Less sensitive to outliers.

Example: Standardization

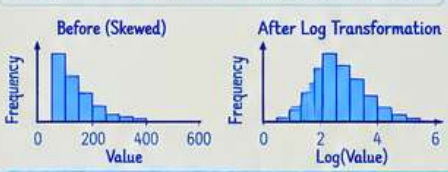
```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
df_scaled = scaler.fit_transform(df[numerical_cols])
```

7. Log Transformation

- Applied to reduce skewness in highly skewed data.
- Helps to make the distribution more normal and reduce the impact of outliers.

Example:

```
import numpy as np
df['log_feature'] = np.log1p(df['feature'])
# log1p(x) = log(1 + x) (handles zero values)
```



8. Date/Time Features

- Extracting useful information from date and time variables.
- Date/time features often contain important patterns like seasonality, trends and cyclic behavior.

Common Date/Time Features

Feature	Description
Year	Extract year from date
Month	Extract month (1-12)
Day	Extract day of month
Day of Week	Extract day of week (0-6)
Is Weekend	Binary feature (1 if weekend)
Quarter	Extract quarter (1-4)
Hour	Extract hour from datetime
Is Holiday	Binary feature (if holiday)

Example: Extracting Date Features

```
df['date'] = pd.to_datetime(df['date'])
df['year'] = df['date'].dt.year
df['month'] = df['date'].dt.month
df['day'] = df['date'].dt.day
df['day_of_week'] = df['date'].dt.dayofweek
df['is_weekend'] = df['day_of_week'] >= 5
```

Example Output

date	year	month	day	day_of_week	is_weekend
2024-01-05	2024	1	5	4	0
2024-01-06	2024	1	6	5	1
2024-01-07	2024	1	7	6	1

9. Domain-Based Features

- Creating features using domain knowledge or business understanding.
- Helps to capture meaningful patterns that may not be obvious from raw data.
- Often leads to significant improvement in model performance.

Examples:
- Price per unit (price / area) in house price prediction.
- Customer tenure in churn prediction.
- Distance from city center in real estate.
- Total number of transactions in fraud detection.
- Average rating per user in recommendation systems.

Machine Learning Complete Notes

6 Linear Regression

- Linear Regression is a supervised learning algorithm used to predict a continuous numerical value.
- It models the linear relationship between the input features (independent variables) and the target variable (dependent variable).



1. What is Regression?

- Regression is a type of supervised learning problem where the target variable is continuous.
- The goal is to find a function that maps input features to a continuous output value.

Examples:
- Predicting house prices
- Predicting sales based on advertisement
- Predicting temperature, demand, etc.

2. Simple vs Multiple Linear Regression

Simple Linear Regression

- Uses one independent variable (feature) to predict the target.
- Equation:

$$\hat{y} = \beta_0 + \beta_1 x$$

Here,
- x → input feature
- β_0 → intercept
- β_1 → coefficient (slope)

Multiple Linear Regression

- Uses two or more independent variables to predict the target.
- Equation:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$

Here,
- $x_1, x_2, \ldots, x_n$ → input features
- β_0 → intercept
- $\beta_1, \beta_2, \ldots, \beta_n$ → coefficients

3. Equation

- The linear regression model can be written as:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$

Where,
- $\hat{y}$ → predicted value
- β_0 → intercept (bias)
- $\beta_1, \beta_2, \ldots, \beta_n$ → coefficients (weights)
- $x_1, x_2, \ldots, x_n$ → input features

4. Coefficients

- Coefficients (β) represent the change in the target variable for a one-unit change in the input feature, keeping other features constant.
- A positive coefficient means a positive relationship.
- A negative coefficient means a negative relationship.

Example:
If $\beta_1 = 5$, a one-unit increase in x increases the predicted value by 5 units (assuming other features remain constant).

5. Intercept

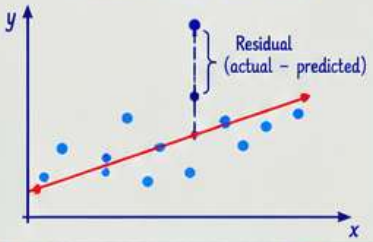
- The intercept (β_0) is the predicted value of the target variable when all input features are zero.
- It represents the starting point of the regression line.

Example:
In house price prediction, the intercept represents the predicted price when all features (e.g., size, bedrooms) are zero (which may not always have a real meaning).

6. Residuals

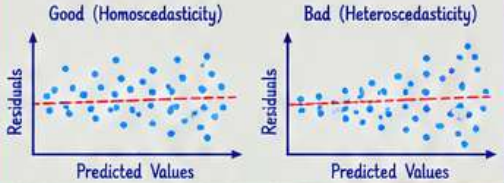
- Residuals are the differences between the actual values and the predicted values.
- They show how well the model is fitting the data.

$$\text{Residual} = y - \hat{y}$$



7. Assumptions of Linear Regression

- **Linearity:** Relationship between input and target is linear.
- **Independence:** Observations are independent of each other.
- **Homoscedasticity:** Constant variance of residuals.
- **Normality:** Residuals are normally distributed.
- **No multicollinearity:** Input features are not highly correlated.



8. Cost Function

- Linear regression uses Mean Squared Error (MSE) as the cost function.
- It measures the average squared difference between actual and predicted values.

$$J(\beta_0, \beta_1, \ldots, \beta_n) = \frac{1}{m} \sum_{i=1}^{m} (y_i - \hat{y}_i)^2$$

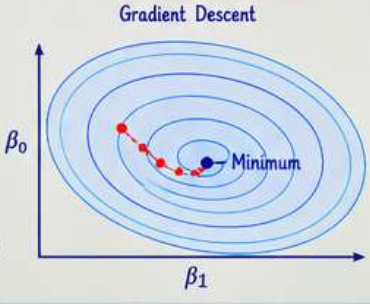
- m → number of training examples
- y_i → actual value
- $\hat{y}_i$ → predicted value

9. Gradient Descent

- Gradient Descent is an optimization algorithm used to minimize the cost function by updating the coefficients iteratively.
- It moves in the direction of the negative gradient (steepest descent).

$$\beta_j = \beta_j - \alpha \frac{\partial J}{\partial \beta_j}$$

- α → learning rate (controls step size)
- The process continues until the cost function converges to a minimum value.



Key Points:
- Start with initial values for coefficients.
- Update coefficients using the gradient of the cost function.
- Repeat until convergence.
- A smaller learning rate gives more precise results but takes longer to converge.

Machine Learning Complete Notes

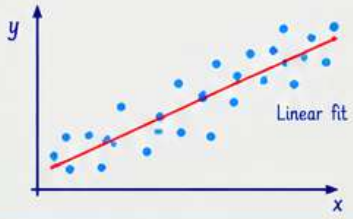
7 Regression Algorithms

- Regression algorithms are used to predict a continuous numerical value.
- Different algorithms make different assumptions and use different techniques to model the relationship between input features and the target variable.

1. Linear Regression

- Models a linear relationship between input features and the target.
- Assumes a straight line can best represent the data.

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$



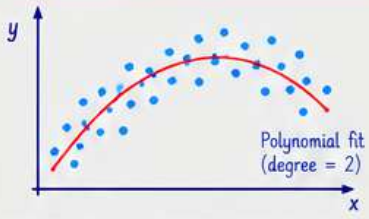
Use case:

- When the relationship is approximately linear.
- Simple and interpretable.

2. Polynomial Regression

- Extends linear regression by adding polynomial terms to capture non-linear relationships.
- Still a form of linear regression with transformed features.

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2 + \cdots + \beta_n x^n$$



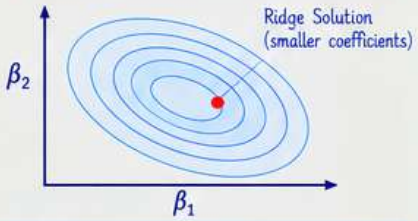
Use case:

- When the relationship is non-linear (curvilinear).
- Useful for simple non-linear patterns.

3. Ridge Regression

- A linear regression model with L2 regularization.
- Helps to reduce overfitting by shrinking the coefficients.

$$J(\beta) = \text{MSE} + \lambda \sum_{j=1}^{n} \beta_j^2$$



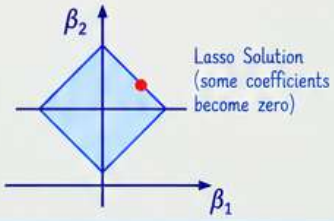
Use case:

- When there are many features or multicollinearity.
- Helps improve model generalization.

4. Lasso Regression

- A linear regression model with L1 regularization.
- Performs feature selection by shrinking some coefficients to exactly zero.

$$J(\beta) = \text{MSE} + \lambda \sum_{j=1}^{n} |\beta_j|$$



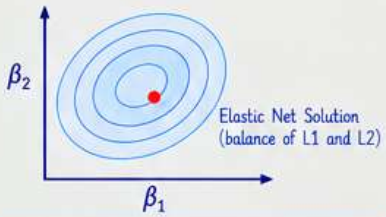
Use case:

- When there are many irrelevant features.
- Useful for feature selection.

5. Elastic Net

- A combination of L1 and L2 regularization.
- Balances feature selection (Lasso) and coefficient shrinkage (Ridge).

$$J(\beta) = \text{MSE} + \lambda_1 \sum |\beta_j| + \lambda_2 \sum \beta_j^2$$

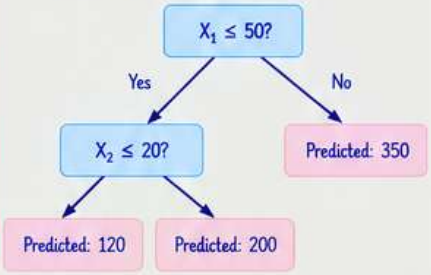


Use case:

- When there are many correlated features.
- Useful when both feature selection and regularization are needed.

6. Decision Tree Regressor

- A non-linear model that splits the data based on feature values.
- Captures complex relationships without assuming linearity.

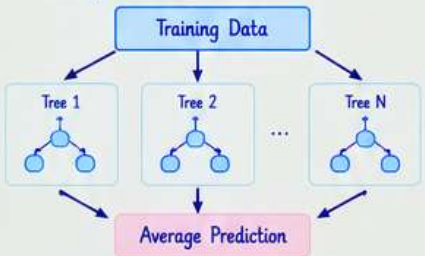


Use case:

- Works well with non-linear data.
- Easy to interpret but can overfit.

7. Random Forest Regressor

- An ensemble of multiple decision trees built on random subsets of data and features.
- The final prediction is the average of all tree predictions.

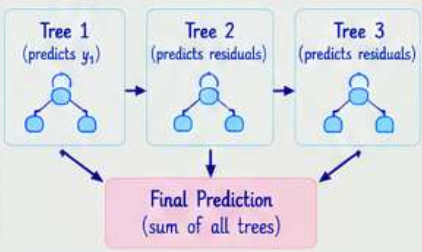


Use case:

- Provides better accuracy and reduces overfitting compared to a single decision tree.
- Works well with large datasets and complex patterns.

8. Gradient Boosting

- Builds trees sequentially, where each new tree corrects the errors of the previous trees.
- Combines weak learners to form a strong model.



Use case:

- Captures complex patterns with high accuracy.
- Can be prone to overfitting if not properly tuned.

9. XGBoost (Extreme Gradient Boosting)

- An advanced and efficient implementation of gradient boosting.
- Uses regularization, parallel processing and other optimizations.

$$\text{Obj} = \underbrace{\sum_{i=1}^{n} l(y_i, \hat{y}_i)}_{\text{Loss function (measure error)}} + \underbrace{\sum_{k=1}^{K} \Omega(f_k)}_{\text{Regularization (prevents overfitting)}}$$

Use case:

- Widely used for structured/tabular data.
- High performance and accuracy.
- Works well in many real-world competitions and applications.

Machine Learning Complete Notes

8 Regression Evaluation

- Regression evaluation metrics are used to measure how well the model's predictions match the actual values.
- They help in comparing different models and selecting the best one.
- Different metrics capture different aspects of prediction error and are useful in different situations.

1. Mean Absolute Error (MAE)

- Measures the average of absolute differences between actual and predicted values.
- It gives equal weight to all errors.
- More interpretable as it is in the same unit as the target variable.

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

Where,

- y_i → actual value
- $\hat{y}_i$ → predicted value
- n → number of observations

2. Mean Squared Error (MSE)

- Measures the average of squared differences between actual and predicted values.
- Penalizes larger errors more than smaller errors.
- Always non-negative.

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Where,

- y_i → actual value
- $\hat{y}_i$ → predicted value
- n → number of observations

3. Root Mean Squared Error (RMSE)

- Square root of MSE.
- Also penalizes larger errors more than smaller errors.
- Interpretable as it is in the same unit as the target variable.

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

Where,

- y_i → actual value
- $\hat{y}_i$ → predicted value
- n → number of observations

4. R^2 (R-squared)

- Represents the proportion of variance in the target variable that is explained by the model.
- Value ranges from 0 to 1 (can be negative if the model performs worse than the mean).
- Higher value indicates better fit.

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

Where,

- y_i → actual value
- $\hat{y}_i$ → predicted value
- $\bar{y}$ → mean of actual values
- n → number of observations

5. Adjusted R^2

- Adjusted R^2 accounts for the number of input features in the model.
- It penalizes the addition of irrelevant features.
- Useful when comparing models with different numbers of features.

$$R^2_{adj} = 1 - \frac{(1 - R^2)(n - 1)}{n - p - 1}$$

Where,

- R^2 → R-squared value
- n → number of observations
- p → number of independent variables (features)

6. When to Use Each Metric?

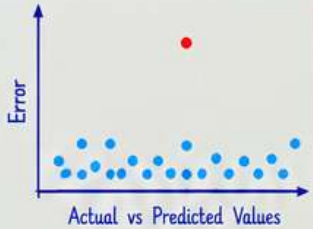
Metric	When to Use
MAE	• When you want a simple and interpretable error metric. • Less sensitive to outliers.
MSE	• When you want to penalize larger errors more. • Useful in optimization and model training (differentiable).
RMSE	• When you want an error metric in the same unit as the target variable. • Commonly used and easy to interpret.
R^2	• To understand how much variance is explained by the model. • Useful for overall model fit.
Adjusted R^2	• When comparing models with different number of features. • Prevents overfitting by penalizing irrelevant features.

7. MAE vs MSE vs RMSE

Aspect	MAE	MSE	RMSE
Formula	$\frac{1}{n}\sum\|y_i - \hat{y}_i\|$	$\frac{1}{n}\sum(y_i - \hat{y}_i)^2$	$\sqrt{\frac{1}{n}\sum(y_i - \hat{y}_i)^2}$
Error treatment	Equal weight to all errors	Penalizes larger errors more (squared)	Penalizes larger errors more (square root of MSE)
Sensitive to outliers	Less sensitive	More sensitive	More sensitive (but less than MSE)
Units	Same as target variable	Squared units (e.g., $€^2$)	Same as target variable
Interpretability	Easy to interpret	Harder to interpret	Easy to interpret
Common use cases	Robust to outliers, real-world applications	Model training, optimization	General evaluation, when interpretable error is needed

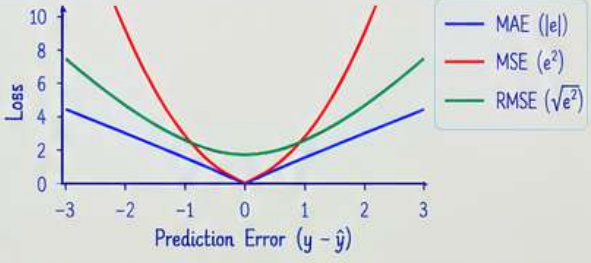
8. Visual Comparison

Effect of Outliers on Metrics



- MAE is less affected by outliers.
- MSE penalizes outliers heavily (squared error).
- RMSE is also affected by outliers but less than MSE.

Same Data, Different Error Functions



Machine Learning Complete Notes

9 Logistic Regression

- Logistic Regression is a supervised learning algorithm used for classification problems.
- It models the probability that a given input belongs to a particular class using the logistic (sigmoid) function.
- Although it is called "regression", it is used for classification because it predicts probabilities (values between 0 and 1).

1. Classification

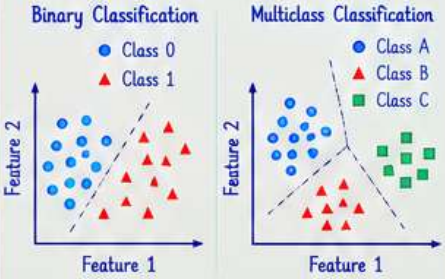
- Classification is a type of supervised learning problem where the target variable is a discrete category or class.
- The model learns from labeled data to predict the class of new data points.

Examples:

- Email spam detection (spam / not spam)
- Disease prediction (disease / no disease)
- Image classification (cat / dog)
- Customer churn prediction (yes / no)

2. Binary vs Multiclass

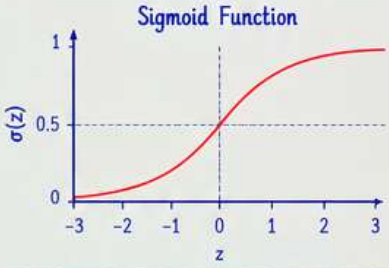
- **Binary Classification:** The target variable has two classes (e.g., 0 and 1, yes and no).
- **Multiclass Classification:** The target variable has more than two classes (e.g., class A, B, C).



3. Sigmoid Function

- Logistic regression uses the sigmoid function to convert the linear output into a probability between 0 and 1.
- The sigmoid function is defined as:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



4. Model Equation

- The logistic regression model computes a linear combination of input features and passes it through the sigmoid function.

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Where,

- $x_1, x_2, \ldots, x_n$ → input features
- β_0 → intercept (bias)
- $\beta_1, \beta_2, \ldots, \beta_n$ → coefficients (weights)
- p → predicted probability (0 to 1)

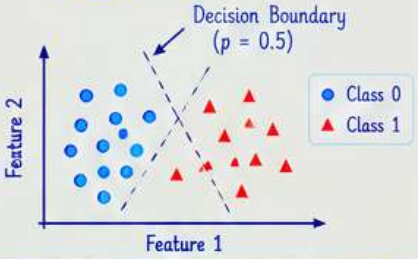
5. Probability

- The output of logistic regression is a probability value between 0 and 1.
- It represents the likelihood of the input belonging to the positive class (class 1).

Example:
If $p = 0.8$, it means there is an 80% chance that the input belongs to class 1.

6. Decision Boundary

- The decision boundary is the line (or hyperplane) that separates the classes.
- For binary classification, it is defined where $p = 0.5$ (i.e., $z = 0$).
- Points on one side of the boundary are classified as class 1 and the other side as class 0.



7. Threshold

- A threshold is used to convert the predicted probability into a class label.
- The default threshold is 0.5, but it can be adjusted based on the problem.

$$\text{Class} = \begin{cases} 1 & \text{if } p \geq \text{threshold} \\ 0 & \text{if } p < \text{threshold} \end{cases}$$

8. Log Loss (Binary Cross-Entropy)

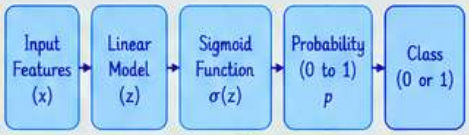
- Log loss measures the difference between the actual labels and the predicted probabilities.
- It penalizes confident but wrong predictions more heavily.

$$J(\beta) = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \log(p_i) + (1 - y_i)\log(1 - p_i)\right]$$

Where,

- y_i → actual label (0 or 1)
- p_i → predicted probability
- n → number of observations

9. Prediction Workflow



$z = \beta_0 + \beta X$ $\quad p = \frac{1}{1 + e^{-z}}$ $\quad$ If $p \geq$ threshold → 1, If $p <$ threshold → 0

10. Example: Spam Detection

Feature	Value (x)	Coefficient (β)	β × x
Intercept	1	-2.0	-2.0
Number of links	3	0.8	2.4
Contains offer (0/1)	1	1.2	1.2
Message length	100	0.01	1.0

$$z = -2.0 + 0.8(3) + 1.2(1) + 0.01(100) = 2.6$$

$$p = \frac{1}{1 + e^{-2.6}} = 0.93$$

If threshold = 0.5 → Predicted class = 1 (Spam)

11. Multiclass Logistic Regression

- For multiclass classification, techniques like One-vs-Rest (OvR) or Softmax are used.

Softmax Function (Multiclass)

$$P(y = k) = \frac{e^{z_k}}{\sum_{j=1}^{k} e^{z_j}}$$

- Outputs a probability for each class.
- The class with the highest probability is chosen as the predicted class.

12. Key Points

- Used for classification problems.
- Outputs probability between 0 and 1 using the sigmoid function.
- Can be used for binary or multiclass classification.
- Decision boundary is defined at $p = 0.5$ (by default).
- Log loss is the cost function.
- Coefficients are learned using optimization algorithms like gradient descent.
- Simple, interpretable and works well for linearly separable data.
- Assumes a linear relationship between input features and the log-odds of the target.

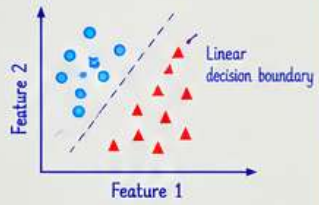
Machine Learning Complete Notes

10 Classification Algorithms

- Classification algorithms are used to predict a discrete class or category for a given input.
- Different algorithms make different assumptions and use different techniques to find the decision boundary between classes.

1. Logistic Regression

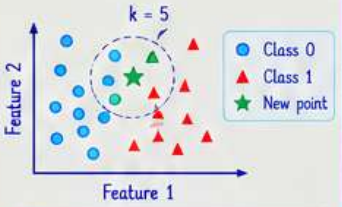
- A linear model that uses the sigmoid function to predict the probability of a class.
- Used for binary and multiclass classification.
- Simple, fast and interpretable.



Use case:
- When the relationship between features and target is (approximately) linear.
- Works well as a baseline model.
- Useful when interpretability is important.

2. K-Nearest Neighbors (KNN)

- A non-parametric, instance-based algorithm.
- Classifies a point based on the majority class among its k nearest neighbors.
- Simple and effective for small to medium datasets.



Use case:
- When the decision boundary is non-linear.
- Works well with small datasets.
- Useful when data has complex patterns.

3. Decision Tree

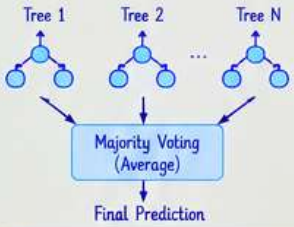
- Splits the data into branches based on feature values.
- Creates a tree-like structure of decision rules.
- Easy to interpret and visualize.



Use case:
- When you need an interpretable model.
- Works with both numerical and categorical data.
- Good for understanding feature importance.

4. Random Forest

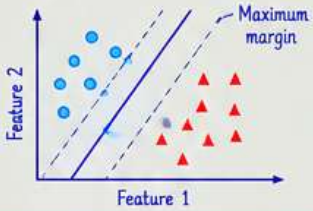
- An ensemble of multiple decision trees built on random subsets of data and features.
- Reduces overfitting compared to a single decision tree.
- Usually gives high accuracy.



Use case:
- When you want high accuracy and robustness.
- Works well with large datasets.
- Handles non-linear relationships effectively.

5. Support Vector Machine (SVM)

- Finds the optimal hyperplane that maximizes the margin between classes.
- Can handle non-linear data using kernel trick (e.g., RBF).
- Effective in high-dimensional spaces.



Use case:
- When the dataset is small to medium sized.
- Works well in high-dimensional spaces.
- Useful when classes are well separated with a clear margin.

6. Naive Bayes

- A probabilistic model based on Bayes' theorem.
- Assumes features are conditionally independent given the class.
- Fast, simple and works well with high-dimensional data.

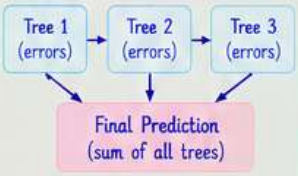
$$P(C \mid X) = \frac{P(X \mid C)P(C)}{P(X)}$$

Where,
- $C \rightarrow$ class
- $X \rightarrow$ input features
- $P(C \mid X) \rightarrow$ posterior probability

Use case:
- Works well with text data (e.g., spam detection).
- Effective for high-dimensional datasets.
- Good when features are mostly independent.

7. Gradient Boosting

- Builds trees sequentially, where each new tree corrects the errors of the previous ones.
- Reduces bias and can capture complex patterns.
- Examples: XGBoost, LightGBM, CatBoost.



Use case:
- When you need high accuracy.
- Works well with structured data.
- Handles complex relationships.
- Can be prone to overfitting if not tuned properly.

8. XGBoost (Extreme Gradient Boosting)

- An optimized implementation of gradient boosting.
- Uses regularization, parallel processing and tree pruning.
- Known for high performance and efficiency.

$$\text{Obj} = \sum_{i=1}^{n} l(y_i, \hat{y}_i) + \sum_{k=1}^{K} \Omega(f_k)$$

Loss function (measure error) — Regularization (prevents overfitting)

Use case:
- When you want the best performance on structured data.
- Works well with large datasets.
- Widely used in machine learning competitions.
- More complex to tune compared to basic models.

Comparison of Classification Algorithms

Algorithm	Type	Decision Boundary	Handles Non-linear Data	Interpretability	Training Speed	Tends to Overfit	Suitable Dataset Size
Logistic Regression	Linear model	Linear	No (unless transformed)	High	Fast	Low	Small – Large
KNN	Instance-based	Non-linear	Yes	Medium	Slow (predicting)	Medium	Small – Medium
Decision Tree	Tree-based	Non-linear	Yes	High	Fast	High	Small – Medium
Random Forest	Ensemble (Bagging)	Non-linear	Yes	Medium	Medium	Low	Medium – Large
SVM	Kernel-based	Linear / Non-linear	Yes (with kernel)	Low	Medium – Slow	Low	Small – Medium
Naive Bayes	Probabilistic	Linear (in feature space)	Limited	High	Fast	Low	Small – Large
Graembnt Boosting	Ensemble (Boosting)	Non-linear	Yes	Low – Medium	Slow	Medium	Medium – Large
XGBoost	Ensemble (Boosting)	Non-linear	Yes	Low – Medium	Medium	Medium	Medium – Large

When Should We Use Which Algorithm?

Situation / Requirement	Recommended Algorithm(s)
Need a simple, interpretable model with linear relationship	Logistic Regression
Small dataset with non-linear patterns	KNN, SVM (with kernel)
Need an explainable model	Decision Tree (or Random Forest for better performance)
Need high accuracy and robustness	Random Forest, Gradient Boosting, XGBoost
High-dimensional data (e.g., text data)	Naive Bayes, SVM
Large dataset with complex patterns	Gradient Boosting, XGBoost
Real-time prediction with fast training	Logistic Regression, Naive Bayes, Decision Tree
When overfitting is a concern	Random Forest, XGBoost (with regularization)
Benchmark or baseline model	Logistic Regression, Naive Bayes
Machine learning competitions / best performance	XGBoost (often), Gradient Boosting

Machine Learning Complete Notes

11 Classification Evaluation

- Classification evaluation metrics are used to measure how well the model predicts the correct class labels.
- Different metrics capture different aspects of model performance, especially in imbalanced datasets.
- It is important to choose the right metric based on the business problem and the type of error that is more costly.

1. Confusion Matrix

- A confusion matrix shows the counts of correct and incorrect predictions for each class.
- It forms the basis for most classification metrics.

Actual Class \ Predicted Class	Positive (1)	Negative (0)
Positive (1)	True Positive (TP) Correctly predicted positive	False Negative (FN) Actual positive but predicted negative
Negative (0)	False Positive (FP) Actual negative but predicted positive	True Negative (TN) Correctly predicted negative

2. TP, TN, FP, FN

- **True Positive (TP):** Actual positive and predicted positive.
- **True Negative (TN):** Actual negative and predicted negative.
- **False Positive (FP):** Actual negative but predicted positive (Type I error).
- **False Negative (FN):** Actual positive but predicted negative (Type II error).

Summary:
- TP → Model correctly predicted the positive class.
- TN → Model correctly predicted the negative class.
- FP → Model incorrectly predicted positive (false alarm).
- FN → Model incorrectly predicted negative (missed detection).

3. Accuracy

- Accuracy is the proportion of total correct predictions.
- Works well only when the classes are balanced.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

4. Precision

- Precision tells us, out of all predicted positives, how many are actually positive.
- High precision means low false positives.

$$\text{Precision} = \frac{TP}{TP + FP}$$

5. Recall (Sensitivity)

- Recall tells us, out of all actual positives, how many are correctly predicted.
- High recall means low false negatives.

$$\text{Recall} = \frac{TP}{TP + FN}$$

6. F1-Score

- F1-score is the harmonic mean of precision and recall.
- Useful when we need a balance between precision and recall.

$$F1 = 2 \times \frac{(\text{Precision} \times \text{Recall})}{\text{Precision} + \text{Recall}}$$

7. Specificity

- Specificity tells us, out of all actual negatives, how many are correctly predicted as negative.
- High specificity means low false positives.

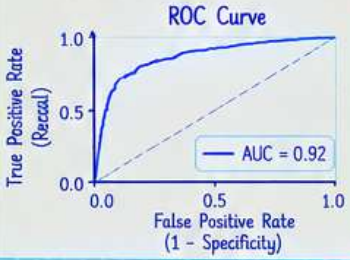
$$\text{Specificity} = \frac{TN}{TN + FP}$$

8. Summary of Metrics

Metric	Formula	What it Measures
Accuracy	$\frac{TP + TN}{TP + TN + FP + FN}$	Overall correctness (works well for balanced data)
Precision	$\frac{TP}{TP + FP}$	How many predicted positives are actually positive
Recall (Sensitivity)	$\frac{TP}{TP + FN}$	How many actual positives are correctly predicted
Specificity	$\frac{TN}{TN + FP}$	How many actual negatives are correctly predicted
F1-Score	$2 \times \frac{(\text{Precision} \times \text{Recall})}{\text{Precision} + \text{Recall}}$	Balance between precision and recall

9. ROC Curve and AUC

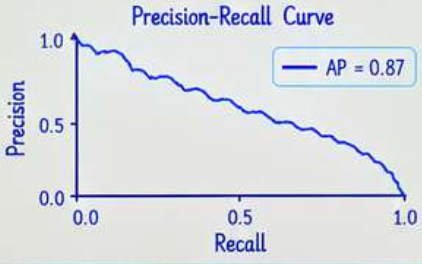
- ROC (Receiver Operating Characteristic) curve plots True Positive Rate (Recall) vs False Positive Rate at different threshold values.
- **AUC** (Area Under the Curve) measures the overall ability of the model to distinguish between classes.



- AUC ranges from 0 to 1.
- AUC = 1 → perfect classification.
- AUC = 0.5 → no better than random guessing.
- Higher AUC indicates better model performance.

10. Precision-Recall Curve

- Plots Precision vs Recall at different threshold values.
- More informative than ROC curve when the dataset is imbalanced.
- A higher area under the curve indicates better performance, especially for the positive class.



11. When to Use Which Metric?

Situation	Recommended Metric(s)
Balanced dataset, general performance	Accuracy, F1-score
Imbalanced dataset (most important)	Precision, Recall, F1-score, ROC-AUC, Precision-Recall Curve
When false positives are costly	Precision, Specificity
When false negatives are costly	Recall (Sensitivity)
Overall model comparison (across thresholds)	ROC-AUC
Focus on positive class performance	Precision-Recall Curve, F1-score
Classification with real-world applications (e.g., medical diagnosis)	Recall, F1-score, ROC-AUC

12. Key Takeaways

- Always look at multiple metrics, not just accuracy.
- Use confusion matrix to understand the model's errors.
- For imbalanced data, focus on precision, recall, F1-score and ROC-AUC.
- Precision-Recall curve is more useful than ROC curve when positive class is rare.
- Choose the metric based on the business problem and the type of error that is more costly.
- A good model should have high precision, high recall and a high AUC (depending on the use case).

12 Decision Trees

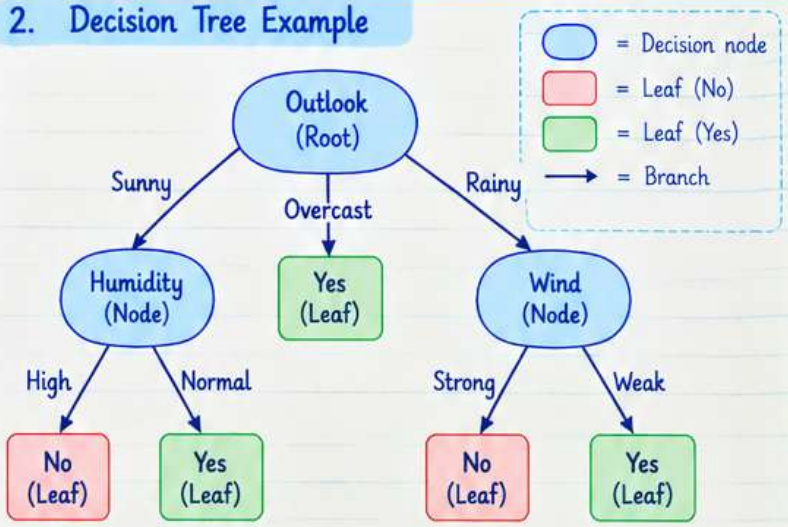
Machine Learning Complete Notes

- A decision tree is a simple and interpretable machine learning algorithm used for both classification and regression problems.
- It splits the data based on feature values to make predictions, forming a tree-like structure.

1. Structure of a Decision Tree

- **Root:** The top most node that represents the entire dataset.
- **Node:** A decision point where the data is split based on a feature.
- **Branch:** The line that connects a node to its child nodes (represents a decision).
- **Leaf:** The end node that gives the final prediction (class label or value).

2. Decision Tree Example



3. Splitting

- The data is split based on a feature and a threshold (for numerical features) or a category (for categorical features).
- The split is chosen such that the resulting subsets are as pure as possible (contain mostly one class).

4. Gini Impurity

- Measures the likelihood of a random sample being misclassified.
- Lower Gini impurity means a purer node.

$$\text{Gini} = 1 - \sum_{i=1}^{n} p_i^2$$

where,
p_i = proportion of class i
n = number of classes

5. Entropy

- Measures the amount of disorder in the data.
- Lower entropy means a purer node.

$$\text{Entropy} = -\sum_{i=1}^{n} p_i \log_2(p_i)$$

where,
p_i = proportion of class i
n = number of classes

6. Information Gain

- Measures the reduction in impurity (Entropy or Gini) after a split.
- The feature with the highest information gain is chosen for splitting.

$$\text{Information Gain} = \text{Impurity(parent)} - \sum_{i=1}^{k} \left\{ \frac{n_i}{n} \text{Impurity(child}_i) \right.$$

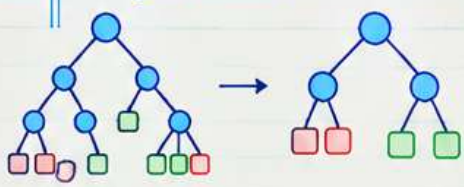
where,
n_i = number of samples in child node
n = total number of samples
k = number of child nodes

7. Overfitting

- A decision tree can become very complex and fit the training data too well, including noise (overfitting).
- This leads to poor performance on unseen data.

8. Pruning

- Pruning removes unnecessary nodes/branches from the tree to reduce overfitting.
- It helps in improving generalization.



Before Pruning (overfitting) → After Pruning (simpler tree)

9. Types of Decision Trees

Classification Tree

- Predicts a categorical output (class label).
- Uses Gini impurity or Entropy for splitting.

Regression Tree

- Predicts a continuous output (numeric value).
- Uses metrics like Mean Squared Error (MSE) for splitting.

10. Key Takeaways

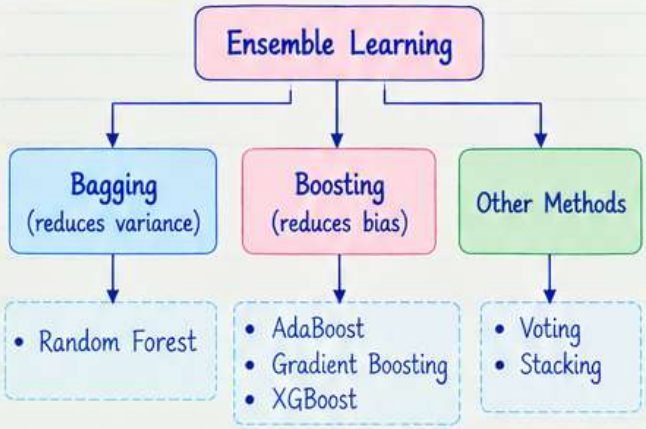
- Simple and easy to interpret.
- Works for both classification
- and regression.
- Can handle non-linear data.
- Prone to overfitting (use pruning).
- Feature scaling is not required.
- Used as a base model in ensemble methods (Random Forest, Boosting).

13 Ensemble Learning

Machine Learning Complete Notes

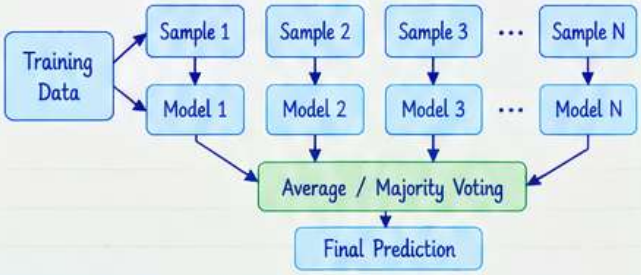
- Ensemble learning combines multiple models (called base learners) to make better predictions than a single model.
- It leverages the wisdom of the crowd — multiple weak learners together form a strong learner.
- It reduces overfitting, improves accuracy and gives more robust and stable predictions.

1. Types of Ensemble Learning



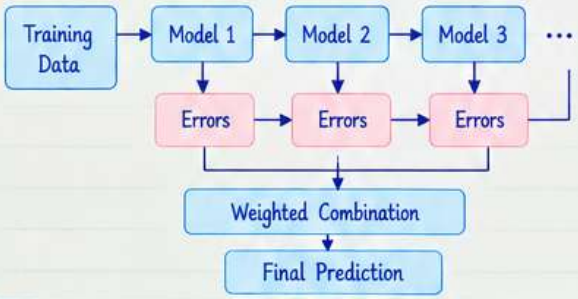
2. Bagging (Bootstrap Aggregating)

- Trains multiple models in parallel using different bootstrap samples (random subsets) of the data.
- Final prediction is obtained by averaging (regression) or majority voting (classification).
- Reduces variance and helps prevent overfitting.



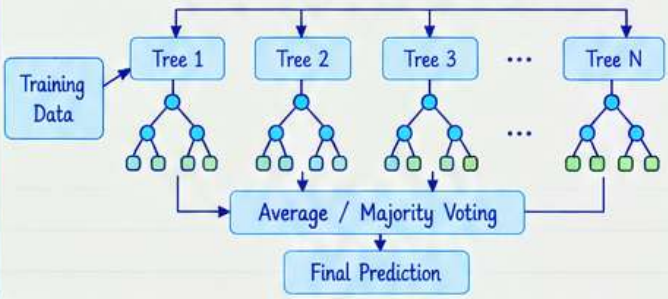
3. Boosting

- Trains models sequentially, where each new model focuses on correcting the errors made by the previous models.
- Combines many weak learners to create a strong learner.
- Reduces bias and can handle complex patterns.



4. Random Forest

- An ensemble of multiple decision trees trained on different bootstrap samples with random feature selection.
- Uses bagging.
- Final prediction is the average (regression) or majority vote (classification) of all trees.
- Reduces overfitting and works well with large datasets.



5. AdaBoost (Adaptive Boosting)

- Sequentially trains weak learners (usually decision trees with single split).
- Each new model focuses more on the misclassified samples.
- Combines models using weighted voting.
- Simple and effective, but sensitive to noise and outliers.

6. Gradient Boosting

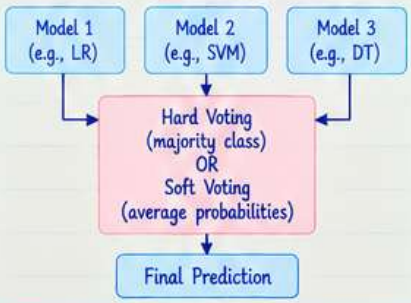
- Builds trees sequentially, where each new model corrects the errors of the previous model using gradient descent.
- Optimizes a differentiable loss function.
- Examples: Gradient Boosting (GBM), XGBoost, LightGBM, CatBoost.
- Generally gives high accuracy but can be computationally expensive.

7. XGBoost (Extreme Gradient Boosting)

- An optimized implementation of gradient boosting.
- Uses regularization (L1 and L2) to prevent overfitting.
- Handles missing values and is highly efficient.
- Known for high performance in competitions and real-world problems.

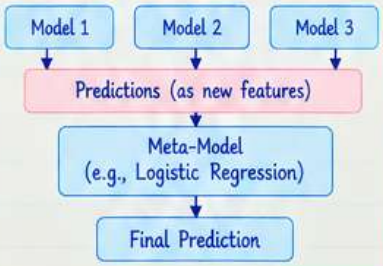
8. Voting (Ensemble Method)

- Combines predictions from multiple models (e.g., Logistic Regression, SVM, Decision Tree).
- Can use hard voting (majority vote) or soft voting (average probabilities).



9. Stacking (Ensemble Method)

- Combines multiple base models and uses a meta-model to learn from their predictions.
- Base models can be different algorithms.
- Usually gives better performance than simple voting.



10. Bagging vs Boosting

Aspect	Bagging	Boosting
Training	Parallel	Sequential
Data sampling	Bootstrap samples (with replacement)	Uses full data (weights updated)
Focus	Reduces variance	Reduces bias
Base learners	Independent models	Dependent models
Combining method	Average / Majority voting	Weighted voting
Overfitting	Less prone to overfitting	Can overfit (especially with noise)
Examples	Random Forest	AdaBoost, Gradient Boosting, XGBoost
Best suited for	High variance models (e.g., Decision Trees)	High bias models

14 Support Vector Machine & KNN

Machine Learning Complete Notes

1. Support Vector Machine (SVM)

- SVM is a powerful supervised learning algorithm used for both classification and regression (SVR).
- It finds the optimal hyperplane that best separates the classes by maximizing the margin between them.
- It works well in high-dimensional spaces and is effective for small to medium-sized datasets.

2. Hyperplane

- A hyperplane is a decision boundary that separates different classes in the feature space.
- In 2D, it is a line; in 3D, a plane; in higher dimensions, a hyperplane.

3. Margin

- The margin is the distance between the hyperplane and the nearest data points (support vectors) from each class.
- SVM chooses the hyperplane with the maximum margin.

$$\text{Margin} = \frac{2}{||w||}$$

4. Support Vectors

- Support vectors are the closest data points to the hyperplane from both classes.
- These points define the position and orientation of the hyperplane.

5. Kernel Trick

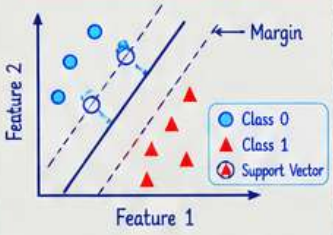
- When data is not linearly separable, SVM uses a kernel function to transform the data into a higher-dimensional space where a linear decision boundary can be found.

6. Common Kernels

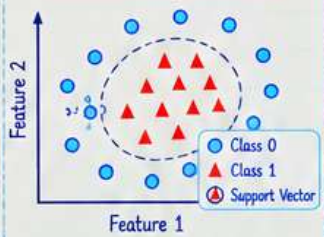
Linear Kernel	RBF (Radial Basis Function) Kernel
$K(x, x') = x \cdot x'$	$K(x, x') = \exp(-\gamma \|\|x - x'\|\|^2)$
• Used when data is linearly separable. • Fast and simple. • Works well with high-dimensional data.	• Used when data is non-linearly separable. • Maps data to higher dimensions. • Most commonly used kernel.

7. Linear vs RBF Kernel (Visual)

Linear Kernel (Linearly Separable)



RBF Kernel (Non-linear)



8. K-Nearest Neighbors (KNN)

- KNN is a non-parametric, instance-based learning algorithm used for both classification and regression.
- It predicts the output based on the majority class (classification) or average (regression) of the k nearest neighbors.
- It is simple, intuitive and works well with small to medium-sized datasets.

9. Distance Metric

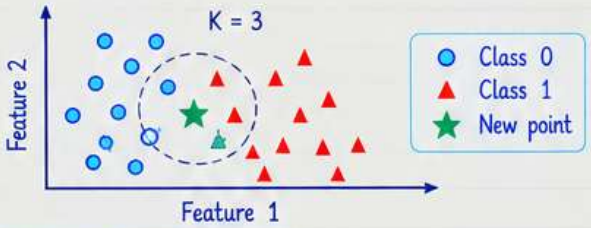
- KNN finds the k closest data points using a distance metric. Common metrics:
  - Euclidean Distance: $d(x, x') = \sqrt{\sum_{i=1}^{n} (x_i - x'_i)^2}$
  - Manhattan Distance: $d(x, x') = \sum_{i=1}^{n} |x_i - x'_i|$
  - Minkowski Distance: $d(x, x') = \left(\sum_{i=1}^{n} |x_i - x'_i|^p\right)^{\frac{1}{p}}$

10. K Value

- K is the number of nearest neighbors to consider.
- A small K can be sensitive to noise (high variance).
- A large K can smooth the decision boundary (high bias).

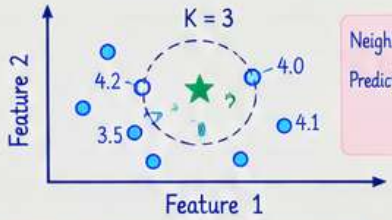
11. KNN Classification

- The class is predicted based on the majority class among the k nearest neighbors.



12. KNN Regression

- The output is predicted as the average of the target values of the k nearest neighbors.



Neighbor values: 4.2, 3.8, 4.0

Predicted value = $\frac{4.2 + 3.8 + 4.0}{3}$ = 4.0

13. Choosing the Right K

- Use cross-validation to find the optimal K.
- Commonly try odd values (e.g., 3, 5, 7, 9).
- Small K → more flexible but sensitive to noise.
- Large K → more stable but may miss local patterns.

14. SVM vs KNN (Comparison)

Aspect	SVM	KNN
Type	Model-based (parametric)	Instance-based (non-parametric)
Working principle	Finds optimal hyperplane	Uses k nearest neighbors
Decision boundary	Maximizes margin	Based on local neighbors
Training time	Higher	Very low (just stores data)
Prediction time	Low	Higher (computes distance)
Works well with	High-dimensional data	Small to medium-sized data
Handling non-linear data	Uses kernel trick (e.g., RBF)	Can capture complex boundaries with appropriate K
Sensitivity to noise	Less sensitive	More sensitive (small K)
Feature scaling	Important	Important
Best use cases	Text data, image data, high-dimensional problems	Simple problems, pattern-based datasets, recommender systerms

16 Dimensionality Reduction

Machine Learning Complete Notes

- Dimensionality reduction transforms high-dimensional data into a lower-dimensional space while preserving the most important information.
- It helps to remove redundant features, reduce noise, and make the data easier to visualize and model.

1. Curse of Dimensionality

- As the number of features (dimensions) increases, the volume of the feature space grows exponentially.
- Data points become sparse, making it difficult to find meaningful patterns.
- Distance metrics become less effective (all points appear almost equally far).
- Requires more data to train models and increases the risk of overfitting.

2. Principal Component Analysis (PCA)

- PCA is a linear dimensionality reduction technique that transforms the original features into a new set of uncorrelated variables called principal components.
- The principal components capture the maximum variance in the data.
- It projects the data onto a lower-dimensional space while retaining the most important information.

3. Principal Components

- Principal components are new features created as linear combinations of the original features.
- The first principal component (PC1) captures the highest variance.
- The second principal component (PC2) captures the next highest variance and is orthogonal to PC1.

4. Variance

- Variance measures how much the data is spread out.
- PCA chooses the directions (principal components) that capture the maximum variance in the data.

$$\text{Variance} = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2$$

Higher variance → more information

5. Eigenvectors and Eigenvalues

- Eigenvectors represent the directions (principal components) in the feature space.
- Eigenvalues represent the amount of variance explained by each eigenvector.
- The eigenvectors are obtained from the covariance matrix of the data.

$$\text{Covariance Matrix: } C = \frac{1}{n-1}X^T X$$

$$Cv = \lambda v$$

λ = eigenvalue (variance explained)
v = eigenvector (principal component)

6. Explained Variance

- Explained variance tells us how much of the total variance is captured by each principal component.
- It is usually expressed as a ratio or percentage.

$$\text{Explained Variance Ratio} = \frac{\lambda_i}{\sum_{j=1}^{n}\lambda_j}$$

where, λ_i = eigenvalue of component i

7. Feature Reduction

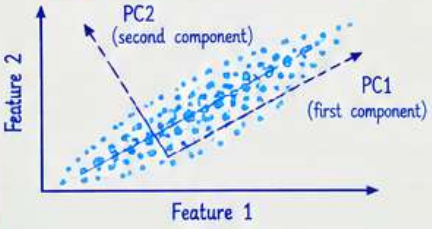
- We can select the top k principal components that capture a desired amount of variance (e.g., 95%).
- This reduces the number of features while retaining the most important information.
- It helps in faster training, reducing overfitting and improving visualization.

8. PCA Workflow

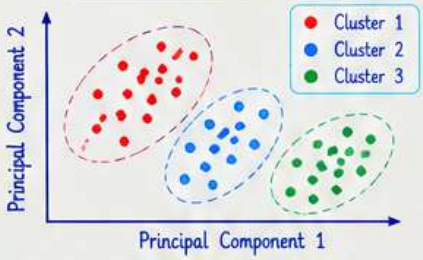
1. Standardize the data (mean = 0, variance = 1)
2. Compute the covariance matrix
3. Calculate eigenvalues and eigenvectors
4. Sort components by eigenvalues (in descending order)
5. Select top k components (based on explained variance)
6. Project the data onto the selected principal components

9. PCA Visualization (2D Projection)

Original High-Dimensional Data (projected to 2D for visualization)



After PCA (Transformed Data)



10. Many Features → PCA → Few Components

Original Data (many features)

	F1	F2	F3	...	F100
x_1	0.5	1.2	3.1	...	0.8
x_2	1.4	0.7	2.9	...	1.1
x_3	0.2	1.8	3.5	...	0.6
...	...	...	...	...	...
x_n	1.1	0.4	2.8	...	1.3

→ **PCA** →

- Find principal components
- Select top k components
- Project the data

Reduced Data (few components)

	PC1	PC2	...	PCk
x_1	2.3	-0.5	...	1.1
x_2	-1.1	2.0	...	-0.3
x_3	0.7	-1.4	...	0.8
...	...	...	...	...
x_n	1.5	0.9	...	-1.2

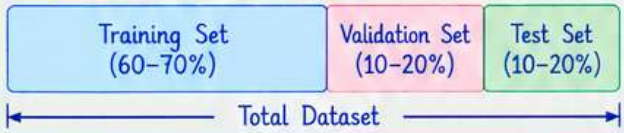
17 Model Selection & Validation

Machine Learning Complete Notes

- Model selection and validation help us choose the best model that generalizes well to unseen data.
- It ensures that the model performs well not only on the training data but also on new, unseen data.

1. Train / Validation / Test Split

- **Training set**: Used to train the model (learn patterns).
- **Validation set**: Used to tune model hyperparameters and select the best model.
- **Test set**: Used to evaluate the final model performance on unseen data.

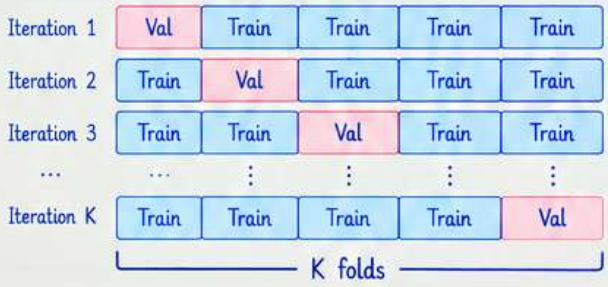


2. Cross-Validation

- Cross-validation is a resampling technique used to evaluate the model performance more reliably.
- The data is split into multiple folds, and the model is trained and validated multiple times.
- It helps to reduce overfitting and gives a more generalized estimate of model performance.

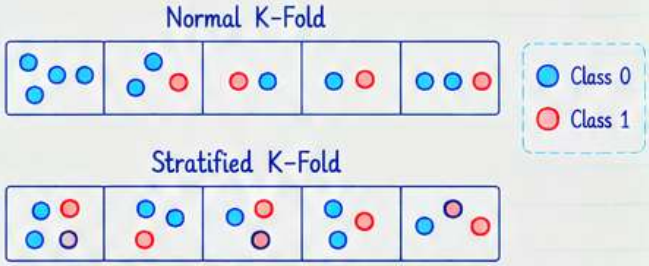
3. K-Fold Cross-Validation

- The data is split into K equal parts (folds).
- The model is trained on $K-1$ folds and validated on the remaining fold.
- This process is repeated K times, and the average performance is taken.



4. Stratified K-Fold Cross-Validation

- Used for classification problems.
- Maintains the same class distribution in each fold as in the original dataset.
- Helps to get a more reliable estimate, especially when the classes are imbalanced.



5. Bias

- Bias is the error due to overly simplified assumptions in the model.
- High bias leads to underfitting.
- The model is too simple to capture the true pattern in the data.

High Bias → Underfitting
(High training error, High test error)

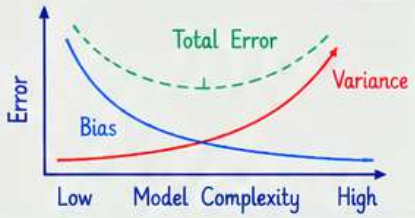
6. Variance

- Variance is the error due to the model being too sensitive to the training data.
- High variance leads to overfitting.
- The model captures noise instead of the true pattern.

High Variance → Overfitting
(Low training error, High test error)

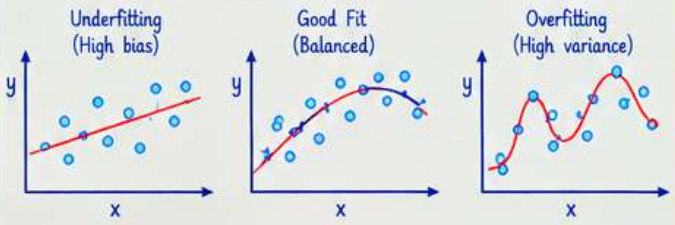
7. Bias-Variance Tradeoff

- As model complexity increases, bias decreases and variance increases.
- The best model is a balance between bias and variance.



8. Underfitting vs Overfitting

Aspect	Underfitting	Good Fit	Overfitting
Model complexity	Too simple	Optimal	Too complex
Training error	High	Low	Very low
Test error	High	Low	High
Bias	High	Low	Low
Variance	Low	Low	High
Captures pattern	No	Yes	Captures noise
Generalization	Poor	Good	Poor



9. Generalization

- Generalization is the ability of the model to perform well on unseen data (test data).
- A good model learns the underlying pattern in the data and not the noise.
- We aim for low test error and good generalization performance.

10. Model Selection

- Choose the model and hyperparameters that give the best performance on the validation set.
- Use cross-validation to compare multiple models.
- Final evaluation is done on the test set only once.
- Consider metrics relevant to the problem (e.g., accuracy, F1-score, RMSE).
- Also consider model complexity, interpretability and computational cost.

18 Hyperparameter Tuning

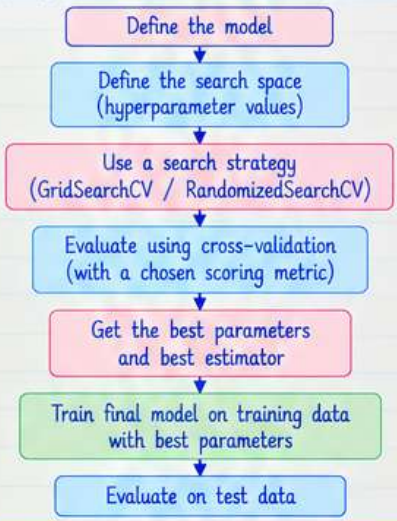
Machine Learning Complete Notes

- Hyperparameter tuning helps us find the best combination of hyperparameters for a model to improve its performance.
- It searches over a predefined search space and uses cross-validation to evaluate the performance of each combination.

1. Parameters vs Hyperparameters

Aspect	Parameters	Hyperparameters
What are they?	Values learned from the data during training.	Values set before training that control the learning process.
Examples	Weights, bias	Learning rate, number of trees, C, max_depth, n_estimators, etc.
How are they obtained?	Automatically learned by the model.	Manually set or searched using techniques like GridSearchCV, RandomizedSearchCV.
Change during training?	Yes	No (fixed before training)

2. Hyperparameter Tuning Process



3. Search Space

- The search space is the set of hyperparameter values to try.
- It can be a fixed list of values (grid) or a range of values (randomized).
- Choosing a good search space is important to get the best model.

Example (for Random Forest):

```
param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [5, 10, 20],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}
```

4. GridSearchCV

- Exhaustively searches all possible combinations in the search space.
- Uses cross-validation to evaluate each combination.
- Finds the best combination based on the chosen scoring metric.
- Works well when the search space is small.

```
from sklearn.model_selection import GridSearchCV

grid = GridSearchCV(
    estimator=model,
    param_grid=param_grid,
    cv=5,
    scoring='accuracy'
)
grid.fit(X_train, y_train)
```

5. RandomizedSearchCV

- Randomly samples a fixed number of combinations from the search space.
- Faster than GridSearchCV.
- Works well when the search space is large.
- Can often find a good set of hyperparameters with fewer iterations.

```
from sklearn.model_selection import RandomizedSearchCV

random_search = RandomizedSearchCV(
    estimator=model,
    param_distributions=param_grid,
    n_iter=50,
    cv=5,
    scoring='accuracy',
    random_state=42
)
random_search.fit(X_train, y_train)
```

6. Cross-Validation

- Cross-validation is used to evaluate each set of hyperparameters more reliably.
- The data is split into K folds, and the model is trained and validated K times.
- The average performance across folds is used to compare different hyperparameter combinations.

7. Scoring

- Scoring is the metric used to evaluate model performance during cross-validation.
- Common scoring metrics:
  - 'accuracy' (classification)
  - 'f1' (classification)
  - 'recall' (classification)
  - 'precision' (classification)
  - 'neg_mean_squared_error' (regression)
  - 'r2' (regression)

8. Best Parameters & Best Estimator

- **best_params_**: gives the best combination of hyperparameters found.
- **best_estimator_**: gives the model trained with those best parameters.
- Use the best_estimator_ to make predictions on test data.

```
print(grid.best_params_)
print(grid.best_estimator_)
# Predict on test data
y_pred =grid.best_estimator_.predict(X_test)
```

9. Why GridSearchCV?

- It systematically evaluates all possible combinations of hyperparameters.
- It uses cross-validation, giving a reliable estimate of model performance.
- It helps in finding the optimal hyperparameters that can significantly improve the model's accuracy.
- It is simple to use and widely used in practice.
- It is effective when the search space is small to medium-sized.

10. Comparison: GridSearchCV vs RandomizedSearchCV

Aspect	GridSearchCV	RandomizedSearchCV
Search strategy	Tries all combinations	Randomly samples combinations
Search space	Fixed list of values	Distributions or list of values
Computation time	Slower	Faster
Best suited for	Small to medium search space	Large search space
Performance	Guarantees best combination (in search space)	Often finds good combination with fewer iterations

19 Advanced ML Concepts

Machine Learning Complete Notes

- These concepts help to build more robust, reliable and interpretable machine learning models.
- They are important in real-world problems where data is noisy, imbalanced and complex.

1. Regularization

- Regularization is used to prevent overfitting by adding a penalty term to the loss function.
- It discourages the model from learning too complex patterns.
- Common types: L1 (Lasso) and L2 (Ridge).

General form :

$$L = Loss + \lambda \times Penalty$$

where, λ = regularization strength (controls the amount of penalty)

2. L1 Regularization → Lasso

- Adds the sum of absolute values of coefficients to the loss function.
- Can shrink some coefficients to exactly zero (feature selection).
- Useful when we want a simpler model with fewer features.

$$L = Loss + \lambda \sum_{i=1}^{n} |w_i|$$

Effect:

- Encourages sparse models (some weights become zero).
- Performs feature selection.

3. L2 Regularization → Ridge

- Adds the sum of squared values of coefficients to the loss function.
- Shrinks the coefficients towards zero but does not make them exactly zero.
- Useful when all features are likely to be important.

$$L = Loss + \lambda \sum_{i=1}^{n} w_i^2$$

Effect:

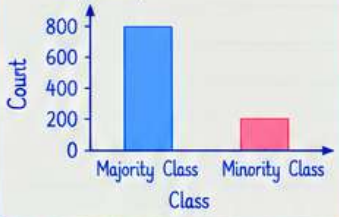
- Reduces large coefficients.
- More stable model.
- Works well with multicollinearity.

4. L1 vs L2 (Comparison)

Aspect	L1 (Lasso)	L2 (Ridge)
Penalty term	$\sum \lvert w_i \rvert$	$\sum w_i^2$
Effect on coefficients	Can be exactly zero	Shrinks towards zero (not zero)
Feature selection	Yes	No
Model	Sparser model	All features retained
When to use	Many irrelevant features	Multicollinearity (highly correlated features)

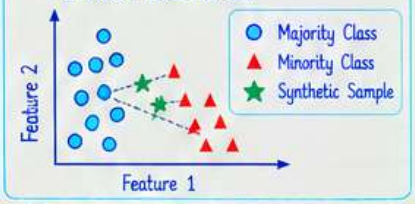
5. Class Imbalance

- Class imbalance occurs when the number of samples in different classes is not equal.
- The model may become biased towards the majority class.
- Common in real-world problems (e.g., fraud detection, medical diagnosis).



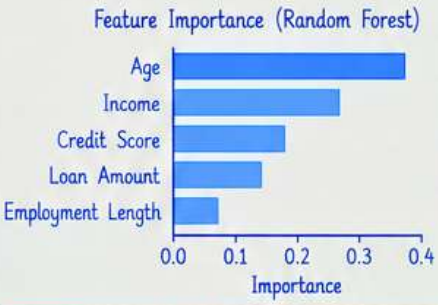
6. SMOTE (Synthetic Minority Over-sampling Technique)

- SMOTE is used to handle class imbalance by generating synthetic samples for the minority class.
- It creates new samples by interpolating between existing minority class samples.
- Helps the model learn better decision boundaries.



7. Feature Importance

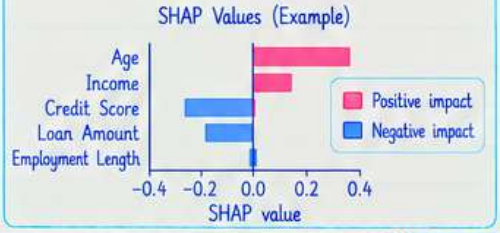
- Feature importance tells us how much each feature contributes to the model's predictions.
- Commonly available in tree-based models (like Random Forest, XGBoost).
- Helps in understanding the most influential features and can be used for feature selection.



8. SHAP Basics

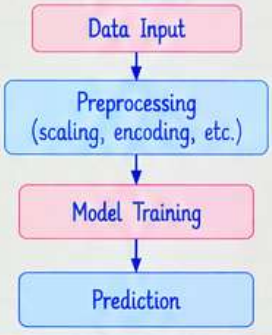
- SHAP (SHapley Additive explanations) explains the contribution of each feature to a particular prediction.
- Based on game theory (Shapley values).
- Provides both global and local interpretability.
- Works with many models (tree-based, linear, etc.).

$$\text{Prediction} = \text{Base value} + \sum \text{SHAP values (feature contributions)}$$



9. Pipelines

- A pipeline chains multiple steps together (e.g., preprocessing, feature engineering, model training) into a single workflow.
- It helps in maintaining a clean and reproducible workflow.
- Commonly used with scikit-learn.



10. Data Leakage

- Data leakage happens when information from outside the training data is used to make predictions.
- It leads to overly optimistic performance and poor generalization on real data.
- Common causes: using future data, including target-related features, improper preprocessing.

Examples:

- Scaling the entire dataset before train-test split.
- Using target information in features (e.g., ID, future values).
- Data from test set leaking into training set.
- Time-series data not split chronologically.

11. Model Interpretability

- Model interpretability helps us understand how the model makes decisions.
- Important for trust, transparency and real-world applications (e.g., healthcare, finance).
- Methods: Feature importance, SHAP, LIME, partial dependence plots, etc.
- Helps in identifying bias, errors and ensuring fair and reliable models.

20 ML Interview Master + Cheat Sheet

Machine Learning Complete Notes

1. Supervised vs Unsupervised Learning

Supervised Learning	Unsupervised Learning
Uses labeled data (target variable)	Uses unlabeled data (no target variable)
Learns a mapping from input to output	Finds hidden patterns and structure in data
Tasks: classification, regression	**Tasks:** clustering, dimensionality reduction, association rule mining
Examples: Linear Regression, Decision Tree, SVM	**Examples:** K-Means, PCA, DBSCAN

2. Classification vs Regression

Classification	Regression
• Predicts discrete categories (class labels) • Output is categorical (e.g., 0/1, A/B/C) • Evaluation metrics: Accuracy, Precision, Recall, F1-score, ROC-AUC • **Examples:** Logistic Regression, Decision Tree, Random Forest, SVM	• Predicts continuous numeric values • Output is continuous (e.g., price, age) • Evaluation metrics: MAE, MSE, RMSE, R^2 score • **Examples:** Linear Regression, Random Forest Regressor, SVR

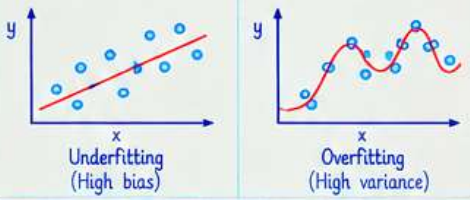
3. Bias vs Variance

- **Bias:** error due to overly simplified assumptions. Leads to underfitting.
- **Variance:** error due to model being too sensitive to training data. Leads to overfitting.

$$\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{Noise}$$

4. Overfitting vs Underfitting

Underfitting	Overfitting
• Model is too simple • High bias, low variance • Poor performance on both training and test data	• Model is too complex • Low bias, high variance • Good performance on training data but poor on test data

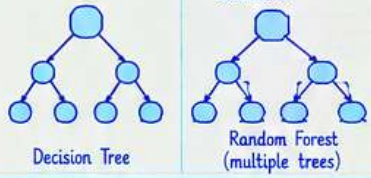


5. Bagging vs Boosting

Bagging	Boosting
• Trains multiple models in parallel • Uses bootstrapped samples (random sampling with replacement) • Reduces variance • **Example:** Random Forest • Generally more robust to overfitting	• Trains models sequentially • Each model focuses on correcting the errors of previous model • Reduces bias • **Examples:** AdaBoost, Gradient Boosting, XGBoost, LightGBM • Can be more prone to overfitting

6. Random Forest vs Decision Tree

Decision Tree	Random Forest
• Single tree model • Easy to interpret • High variance (prone to overfitting) • Captures non-linear relationships	• Ensemble of multiple decision trees • Uses bagging • More stable and generalizes better • Feature importance available



7. L1 vs L2 Regularization

L1 (Lasso)	L2 (Ridge)
• Adds sum of absolute values of coefficients • Can make some coefficients exactly zero (feature selection) • Useful for simpler models with fewer features	• Adds sum of squared values of coefficients • Shrinks coefficients towards zero (not zero) • Works well with multicollinearity • Uses all features
$L = Loss + \lambda \sum_{i=1}^{n} \lvert w_i \rvert$	$L = Loss + \lambda \sum_{i=1}^{n} w_i^2$

8. Precision vs Recall

- **Precision:** proportion of predicted positives that are actually positive.

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall:** proportion of actual positives that are correctly predicted.

$$\text{Recall} = \frac{TP}{TP + FN}$$

- High precision → less false positives
- High recall → less false negatives

9. MAE vs RMSE

- **MAE (Mean Absolute Error):** Average of absolute differences between predicted and actual values.

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^{n} \lvert y_i - \hat{x}_i \rvert$$

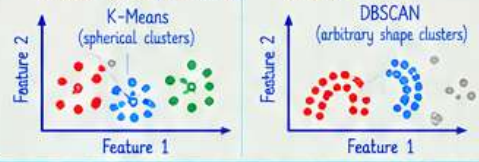
- **RMSE (Root Mean Squared Error):** Square root of average squared differences. Penalizes large errors more.

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{x}_i)^2}$$

- Use MAE when you want equal penalty for all errors.
- Use RMSE when large errors are more critical.

10. K-Means vs DBSCAN

K-Means	DBSCAN
• Partition data into K clusters • Requires predefining K • Works well for spherical clusters • Sensitive to outliers • Uses centroid and distance (usually Euclidean)	• Density-based clustering • Does not require K • Can find clusters of arbitrary shape • Handles noise and outliers • Requires two parameters: eps (radius) and min_samples • Works well for non-spherical clusters



11. Parameter vs Hyperparameter

Parameter	Hyperparameter
• Values learned from data during training • Determined automatically by model • Examples: weights, biases, coefficients	• Set before training • Control the learning process • Chosen using techniques like GridSearchCV, RandomizedSearchCV • Examples: learning rate, max_depth, C, n_estimators

12. GridSearchCV

- Exhaustively searches all possible combinations of hyperparameters from a predefined search space.
- Uses cross-validation to evaluate each combination.
- Returns the best parameters and best estimator.

```
from sklearn.model_selection import GridSearchCV
param_grid = {'C': [0.1, 1, 10],
              'kernel': ['linear', 'rbf']}
grid = GridSearchCV(model, param_grid, cv=5,
                    scoring='accuracy')
grid.fit(X_train, y_train)
```

13. Cross-validation

- Split data into multiple folds.
- Train on K-1 folds and validate on the remaining fold.
- Repeat K times and take the average performance.
- Helps to get a more reliable estimate and reduces overfitting.

5-Fold Cross-Validation

Split 1	Val	Train	Train	Train	Train
Split 2	Train	Val	Train	Train	Train
Split 3	Train	Train	Val	Train	Train
Split 4	Train	Train	Train	Val	Train
Split 5	Train	Train	Train	Train	Val

14. Feature Selection vs Feature Extraction

Feature Selection	Feature Extraction
• Selects a subset of original features • Removes less important features • Examples: filter methods (correlation), wrapper methods (RFE), tree-based importance	• Transforms original features into new features • Reduces dimensionality while retaining important information • Examples: PCA, LDA, Autoencoders, TF-IDF (for text)

15. Data Leakage

- Data leakage happens when information from outside the training data is used to make predictions.
- Leads to overly optimistic performance and poor generalization.
- Common causes: using future data, target-related features, improper preprocessing (e.g., scaling before split).
- Solution: fit all preprocessing steps only on training data (use pipelines).

16. ML Project Workflow

Problem Understanding → Data Collection → EDA & Preprocessing → Feature Selection / Engineering → Model Training → Evaluation & Tuning → Deployment / Reporting